

Network Lab Experiments

1. TCP Connection

Filename: client/client.c

```
1 // Client side code for a tcp connection
2
3 #include <unistd.h>
4 #include <stdlib.h>
5 #include <stdio.h>
6 #include <string.h>
7 #include <sys/stat.h>
8 #include <sys/types.h>
9 #include <sys/socket.h>
10 #include <netinet/in.h>
11 #include <arpa/inet.h>
12
13 int main(){
14     struct sockaddr_in client,server;
15     int client_conn, client_size, sockfd;
16
17     char buff_cli[50] = "", buff_serv[50] = "";
18
19     sockfd = socket(AF_INET, SOCK_STREAM, 0);
20
21     if(sockfd < 0){
22         perror("Socket failed");
23         exit(0);
24     }
25
26     server.sin_family = AF_INET;
27     server.sin_port = 8000;
28     server.sin_addr.s_addr = htonl(INADDR_ANY);
29
30     while(connect(sockfd, (struct sockaddr *) &server, sizeof(server)) < 0){
31         perror("Connection failed, trying again in 10 sec...");
32         sleep(10);
33     }
34
35     for(;;){
36         printf("\nClient : \t");
37         scanf("%s", buff_cli);
38         if(strcmp(buff_cli, "end") == 0) break;
39         send(sockfd, buff_cli, sizeof(buff_cli), 0);
40
41         recv(sockfd, buff_serv, sizeof(buff_serv), 0);
42         if(strcmp(buff_serv, "end") == 0) break;
43         printf("\nServer : \t%s", buff_serv);
44     }
45
46     close(sockfd);
47 }
```

1. TCP Connection

Filename: server/server.c

```
1 // Server side code for tcp connection in c.
2
3 #include <unistd.h>
4 #include <stdlib.h>
5 #include <stdio.h>
6 #include <string.h>
7 #include <sys/stat.h>
8 #include <sys/types.h>
9 #include <sys/socket.h>
10 #include <netinet/in.h>
11 #include <arpa/inet.h>
12 #include "../functions/paliandrome.h"
13
14
15 int main(){
16     struct sockaddr_in client, server;
17     int client_conn, client_size, sock_fd;
18
19     char buff_one[50] = "", buff_two[50] = "";
20
21     sock_fd = socket(AF_INET, SOCK_STREAM, 0);
22
23     if(sock_fd == -1){
24         perror("Socket failed");
25         exit(0);
26     }
27
28     server.sin_family = AF_INET;
29     server.sin_addr.s_addr = htonl(INADDR_ANY);
30     server.sin_port = 8000;
31     if(bind(sock_fd, (struct sockaddr *) &server, sizeof server) < 0){
32         perror("Bind Failed");
33         exit(0);
34     }
35
36     if(listen(sock_fd, 5) < 0) perror("Listen failed");
37     printf("Server is listening\n");
38
39     client_size = sizeof(client);
40
41     client_conn = accept(sock_fd, (struct sockaddr *) &client, &client_size);
42
43
44     for(;;){
45         if(recv(client_conn, buff_one, sizeof(buff_one), 0) < 0){
46             perror("recv failed");
47             exit(0);
48         }
49
50         if(strstr(buff_one, "paliandrom()") != NULL){
51
52             int n = 11;
53             int num = 0;
54
55             while(buff_one[n] != ' '){
56                 num = (num * 10) + (buff_one[n] - '0');
57                 n++;
58             }
59             printf("%d", num);
60             char* msg = pal_fun(num);
61             printf("%s", msg);
62             // if(flag == 1) {
63             //     strcpy(buff_two, "pal");
64             // }
65             // else if(flag == -1) {
66             //     strcpy(buff_two, "!pal");
67             // }
68             strcpy(buff_two, msg);
```

```

69         send(client_conn, buff_two, sizeof(buff_two), 0);
70
71     }
72
73     else if(strcmp(buff_two, "end") == 0){
74         break;
75     }
76     else{
77         printf("\nClient : \t%s", buff_one);
78         printf("\nServer : \t");
79         fgets(buff_two, sizeof(buff_two), stdin);
80         send(client_conn, buff_two, sizeof(buff_two), 0);
81     }
82 }
83
84 close(client_conn);
85 close(sock_fd);
86
87
88 }

```

2. Paliandrome with TCP

Filename: functions/paliandrome.h

```
1 #ifndef PALIANDROME_H
2 #define PALIANDROME_H
3
4 #include <stdio.h>
5 #include <stdlib.h>
6
7
8 char* pal_fun(int num){
9     int rev = 0;
10    int rem = 0;
11    int temp = num;
12
13    while( temp != 0){
14        rem = temp % 10;
15        rev = rev * 10 + rem; //rev = rev * 10 + rem;
16        temp = temp / 10;
17    }
18
19    if( rev == num ){
20        return "pal";
21    }
22    else{
23        return "not pal";
24    }
25 }
26
27 #endif
```

3. UDP connection

Filename: client/udp_client.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7
8 #define PORT 8080
9 #define BUFFER_SIZE 1024
10
11 int main() {
12
13     int sockfd;
14     struct sockaddr_in server_addr;
15     char buffer[BUFFER_SIZE];
16     socklen_t addr_len;
17     int n;
18
19     // 1) Create socket
20     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
21     if (sockfd < 0) {
22         perror("Socket creation failed");
23         exit(EXIT_FAILURE);
24     }
25
26     memset(&server_addr, 0, sizeof(server_addr));
27
28     server_addr.sin_family = AF_INET;
29     server_addr.sin_port = htons(PORT);
30     server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
31
32     addr_len = sizeof(server_addr);
33
34     while (1) {
35
36         // 2) Read message from stdin
37         printf("Enter message: ");
38         fgets(buffer, BUFFER_SIZE, stdin);
39
40         sendto(sockfd, buffer, strlen(buffer), 0,
41              (struct sockaddr*)&server_addr, addr_len);
42
43         // 3) Read message from server
44         n = recvfrom(sockfd, buffer, BUFFER_SIZE - 1, 0,
45              (struct sockaddr*)&server_addr, &addr_len);
46
47         if (n < 0) {
48             perror("recvfrom failed");
49             continue;
50         }
51
52         buffer[n] = '\0';
53
54         printf("Server reply: %s\n", buffer);
55     }
56
57     close(sockfd);
58
59     return 0;
60 }
```

3. UDP connection

Filename: server/udp_server.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7
8 #define PORT 8080
9 #define BUFFER_SIZE 1024
10
11 int main() {
12
13     int sockfd;
14     struct sockaddr_in server_addr, client_addr;
15     char buffer[BUFFER_SIZE];
16     socklen_t addr_len;
17     int n;
18
19     // 0) Create socket
20     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
21     if (sockfd < 0) {
22         perror("Socket creation failed");
23         exit(EXIT_FAILURE);
24     }
25
26     memset(&server_addr, 0, sizeof(server_addr));
27     memset(&client_addr, 0, sizeof(client_addr));
28
29     // 1) Bind IP and port
30     server_addr.sin_family = AF_INET;
31     server_addr.sin_port = htons(PORT);
32     server_addr.sin_addr.s_addr = INADDR_ANY;
33
34     if (bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
35         perror("Bind failed");
36         exit(EXIT_FAILURE);
37     }
38
39     addr_len = sizeof(client_addr);
40
41     printf("UDP server listening on port %d...\n", PORT);
42
43     while (1) {
44
45         // 2) Read message from client
46         n = recvfrom(sockfd, buffer, BUFFER_SIZE - 1, 0,
47                     (struct sockaddr*)&client_addr, &addr_len);
48
49         if (n < 0) {
50             perror("recvfrom failed");
51             continue;
52         }
53
54         buffer[n] = '\0';
55
56         printf("Client says: %s\n", buffer);
57
58         // 3) Send message back to client
59         char reply[] = "Message received";
60
61         sendto(sockfd, reply, strlen(reply), 0,
62               (struct sockaddr*)&client_addr, addr_len);
63     }
64
65     // 4) Close socket (never reached unless server stops)
66     close(sockfd);
67
68     return 0;
69 }
```


4. Sliding window protocol - Stop and Wait

Filename: sliding-window/stop-and-wait/stopandwaitcli.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define PORT 8080
8 #define FRAME_SIZE 2
9
10 int main()
11 {
12     int sockfd;
13     struct sockaddr_in server_addr;
14     socklen_t addr_len = sizeof(server_addr);
15
16     char message[1024];
17     char frame[FRAME_SIZE];
18     int ack;
19
20     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
21
22     server_addr.sin_family = AF_INET;
23     server_addr.sin_port = htons(PORT);
24     server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
25
26     printf("Enter message: ");
27     scanf("%s", message);
28
29     int len = strlen(message);
30
31     printf("\nStarting frame transmission\n\n");
32
33     for(int i=0; i<len; i+=2)
34     {
35         frame[0] = message[i];
36         frame[1] = (i+1 < len) ? message[i+1] : '_';
37
38         printf("Sending frame: %c%c\n", frame[0], frame[1]);
39
40         sendto(sockfd, frame, FRAME_SIZE, 0,
41               (struct sockaddr*)&server_addr, addr_len);
42
43         recvfrom(sockfd, &ack, sizeof(int), 0,
44                 (struct sockaddr*)&server_addr, &addr_len);
45
46         printf("ACK received\n\n");
47
48         sleep(1);
49     }
50
51     char end[2] = "-9";
52
53     sendto(sockfd, end, 2, 0,
54           (struct sockaddr*)&server_addr, addr_len);
55
56     printf("Transmission finished\n");
57
58     close(sockfd);
59
60     return 0;
61 }
```

4. Sliding window protocol - Stop and Wait

Filename: sliding-window/stop-and-wait/stopandwaiterv.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define PORT 8080
8 #define FRAME_SIZE 2
9 #define MAX_MSG 1024
10
11 int main()
12 {
13     int sockfd;
14     struct sockaddr_in server_addr, client_addr;
15     socklen_t addr_len = sizeof(client_addr);
16
17     char frame[FRAME_SIZE + 1];
18     char message[MAX_MSG];
19     int index = 0;
20
21     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
22
23     server_addr.sin_family = AF_INET;
24     server_addr.sin_port = htons(PORT);
25     server_addr.sin_addr.s_addr = INADDR_ANY;
26
27     bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
28
29     printf("Server started. Waiting for frames...\n\n");
30
31     while(1)
32     {
33         recvfrom(sockfd, frame, FRAME_SIZE, 0,
34                 (struct sockaddr*)&client_addr, &addr_len);
35
36         frame[FRAME_SIZE] = '\0';
37
38         if(strcmp(frame, "-9")==0) // first part of -99
39         {
40             printf("\nTermination frame received\n");
41             break;
42         }
43
44         printf("Frame received: %s\n", frame);
45
46         message[index++] = frame[0];
47         message[index++] = frame[1];
48
49         sleep(1); // simulate delay
50
51         int ack = 1;
52
53         sendto(sockfd, &ack, sizeof(int), 0,
54               (struct sockaddr*)&client_addr, addr_len);
55
56         printf("ACK sent\n\n");
57     }
58
59     message[index] = '\0';
60
61     printf("\nConstructed message: %s\n", message);
62
63     close(sockfd);
64
65     return 0;
66 }
```

4. Sliding window protocol - Go Back N

Filename: sliding-window/go-back-n/gobackncli.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define PORT 8080
8 #define FRAME_SIZE 2
9 #define WINDOW_SIZE 4
10
11 typedef struct {
12     int seq;
13     char data[FRAME_SIZE];
14 } frame;
15
16 int main()
17 {
18     int sockfd;
19     struct sockaddr_in server_addr;
20     socklen_t addr_len = sizeof(server_addr);
21
22     char message[1024];
23
24     frame frames[512];
25
26     int ack;
27
28     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
29
30     server_addr.sin_family = AF_INET;
31     server_addr.sin_port = htons(PORT);
32     server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
33
34     printf("Enter message: ");
35     scanf("%s", message);
36
37     int len = strlen(message);
38
39     int total_frames = (len + 1) / 2;
40
41     for(int i=0; i<total_frames; i++)
42     {
43         frames[i].seq = i;
44
45         frames[i].data[0] = message[i*2];
46         frames[i].data[1] = (i*2+1 < len) ? message[i*2+1] : '_';
47     }
48
49     int base = 0;
50     int next_seq = 0;
51
52     printf("\nStarting Go-Back-N transmission\n\n");
53
54     while(base < total_frames)
55     {
56         while(next_seq < base + WINDOW_SIZE && next_seq < total_frames)
57         {
58             printf("Sending Frame %d [%c%c]\n",
59                 frames[next_seq].seq,
60                 frames[next_seq].data[0],
61                 frames[next_seq].data[1]);
62
63             sendto(sockfd, &frames[next_seq], sizeof(frame), 0,
64                 (struct sockaddr*)&server_addr, addr_len);
65
66             next_seq++;
67         }
68     }
```

```

69     recvfrom(sockfd,&ack,sizeof(int),0,
70             (struct sockaddr*)&server_addr,&addr_len);
71
72     printf("ACK %d received\n\n",ack);
73
74     base = ack + 1;
75
76     sleep(1);
77 }
78
79 frame end;
80 end.seq = -99;
81
82 sendto(sockfd,&end,sizeof(end),0,
83        (struct sockaddr*)&server_addr,addr_len);
84
85 printf("Transmission complete\n");
86
87 close(sockfd);
88
89 return 0;
90 }

```

4. Sliding window protocol - Go Back N

Filename: sliding-window/go-back-n/gobacknserv.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define PORT 8080
8 #define FRAME_SIZE 2
9 #define MAX_MSG 1024
10
11 typedef struct {
12     int seq;
13     char data[FRAME_SIZE];
14 } frame;
15
16 int main()
17 {
18     int sockfd;
19     struct sockaddr_in server_addr, client_addr;
20     socklen_t addr_len = sizeof(client_addr);
21
22     frame f;
23     int ack;
24
25     int expected = 0;
26
27     char message[MAX_MSG];
28     int index = 0;
29
30     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
31
32     server_addr.sin_family = AF_INET;
33     server_addr.sin_port = htons(PORT);
34     server_addr.sin_addr.s_addr = INADDR_ANY;
35
36     bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
37
38     printf("GBN Server started\n\n");
39
40     while(1)
41     {
42         recvfrom(sockfd, &f, sizeof(f), 0,
43                 (struct sockaddr*)&client_addr, &addr_len);
44
45         if(f.seq == -99)
46             break;
47
48         printf("Frame %d received [%c%c]\n", f.seq, f.data[0], f.data[1]);
49
50         if(f.seq == expected)
51         {
52             message[index++] = f.data[0];
53             message[index++] = f.data[1];
54
55             ack = f.seq;
56             expected++;
57
58             printf("Frame accepted\n");
59         }
60         else
61         {
62             ack = expected - 1;
63             printf("Frame discarded (out of order)\n");
64         }
65
66         sendto(sockfd, &ack, sizeof(int), 0,
67               (struct sockaddr*)&client_addr, addr_len);
68     }
```

```
69     printf("ACK %d sent\n\n",ack);
70 }
71
72 message[index] = '\0';
73
74 printf("Reconstructed message: %s\n",message);
75
76 close(sockfd);
77
78 return 0;
79 }
```

4. Sliding window protocol - Selective Repeat

Filename: sliding-window/selective-repeat/selectiverepeatcli.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define PORT 8080
8 #define FRAME_SIZE 2
9 #define WINDOW_SIZE 4
10
11 typedef struct {
12     int seq;
13     char data[FRAME_SIZE];
14 } frame;
15
16 int main()
17 {
18     int sockfd;
19     struct sockaddr_in server_addr;
20     socklen_t addr_len = sizeof(server_addr);
21
22     char message[1024];
23
24     frame frames[512];
25
26     int acked[512] = {0};
27
28     int ack;
29
30     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
31
32     server_addr.sin_family = AF_INET;
33     server_addr.sin_port = htons(PORT);
34     server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
35
36     printf("Enter message: ");
37     scanf("%s", message);
38
39     int len = strlen(message);
40
41     int total_frames = (len + 1) / 2;
42
43     for(int i=0; i<total_frames; i++)
44     {
45         frames[i].seq = i;
46
47         frames[i].data[0] = message[i*2];
48         frames[i].data[1] = (i*2+1 < len) ? message[i*2+1] : '_';
49     }
50
51     int base = 0;
52     int next_seq = 0;
53
54     printf("\nStarting Selective Repeat transmission\n\n");
55
56     while(base < total_frames)
57     {
58         while(next_seq < base + WINDOW_SIZE && next_seq < total_frames)
59         {
60             printf("Sending Frame %d [%c%c]\n",
61                 frames[next_seq].seq,
62                 frames[next_seq].data[0],
63                 frames[next_seq].data[1]);
64
65             sendto(sockfd, &frames[next_seq], sizeof(frame), 0,
66                 (struct sockaddr*)&server_addr, addr_len);
67
68             next_seq++;
```

```

69     }
70
71     recvfrom(sockfd,&ack,sizeof(int),0,
72             (struct sockaddr*)&server_addr,&addr_len);
73
74     printf("ACK %d received\n",ack);
75
76     acked[ack] = 1;
77
78     while(acked[base])
79         base++;
80
81     sleep(1);
82     printf("\n");
83 }
84
85 frame end;
86 end.seq = -99;
87
88 sendto(sockfd,&end,sizeof(end),0,
89        (struct sockaddr*)&server_addr,addr_len);
90
91 printf("Transmission complete\n");
92
93 close(sockfd);
94
95 return 0;
96 }

```


4. Sliding window protocol - Selective Repeat

Filename: sliding-window/selective-repeat/selectiverepeatserv.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define PORT 8080
8 #define FRAME_SIZE 2
9 #define WINDOW_SIZE 4
10 #define MAX_MSG 1024
11
12 typedef struct {
13     int seq;
14     char data[FRAME_SIZE];
15 } frame;
16
17 int main()
18 {
19     int sockfd;
20     struct sockaddr_in server_addr, client_addr;
21     socklen_t addr_len = sizeof(client_addr);
22
23     frame f;
24     int ack;
25
26     int base = 0;
27
28     frame buffer[WINDOW_SIZE];
29     int received[WINDOW_SIZE] = {0};
30
31     char message[MAX_MSG];
32     int index = 0;
33
34     sockfd = socket(AF_INET, SOCK_DGRAM, 0);
35
36     server_addr.sin_family = AF_INET;
37     server_addr.sin_port = htons(PORT);
38     server_addr.sin_addr.s_addr = INADDR_ANY;
39
40     bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
41
42     printf("Selective Repeat Server started\n\n");
43
44     while(1)
45     {
46         recvfrom(sockfd, &f, sizeof(f), 0,
47                 (struct sockaddr*)&client_addr, &addr_len);
48
49         if(f.seq == -99)
50             break;
51
52         printf("Frame %d received [%c%c]\n", f.seq, f.data[0], f.data[1]);
53
54         if(f.seq >= base && f.seq < base + WINDOW_SIZE)
55         {
56             int index_buf = f.seq % WINDOW_SIZE;
57
58             buffer[index_buf] = f;
59             received[index_buf] = 1;
60
61             ack = f.seq;
62
63             printf("Frame buffered\n");
64         }
65         else
66         {
67             ack = f.seq;
68             printf("Frame outside window\n");
```

```

69     }
70
71     sendto(sockfd,&ack,sizeof(int),0,
72           (struct sockaddr*)&client_addr,addr_len);
73
74     printf("ACK %d sent\n\n",ack);
75
76     while(received[base % WINDOW_SIZE])
77     {
78         frame temp = buffer[base % WINDOW_SIZE];
79
80         message[index++] = temp.data[0];
81         message[index++] = temp.data[1];
82
83         received[base % WINDOW_SIZE] = 0;
84
85         base++;
86     }
87 }
88
89 message[index] = '\0';
90
91 printf("Reconstructed message: %s\n",message);
92
93 close(sockfd);
94
95 return 0;
96 }

```

5. DVR algorithm in C

Filename: DVR/dvralgorithm.c

```
1 #include <stdio.h>
2
3 #define MAX 10
4 #define INF 999
5
6 int main()
7 {
8     int cost[MAX][MAX];
9     int dist[MAX][MAX];
10    int next[MAX][MAX];
11
12    int n;
13    int i,j,k;
14
15    printf("Enter number of routers: ");
16    scanf("%d",&n);
17
18    printf("Enter cost matrix:\n");
19
20    for(i=0;i<n;i++)
21    {
22        for(j=0;j<n;j++)
23        {
24            scanf("%d",&cost[i][j]);
25
26            dist[i][j] = cost[i][j];
27
28            if(i == j || cost[i][j] == INF)
29                next[i][j] = -1;
30            else
31                next[i][j] = j;
32        }
33    }
34
35    for(k=0;k<n;k++)
36    {
37        for(i=0;i<n;i++)
38        {
39            for(j=0;j<n;j++)
40            {
41                if(dist[i][j] > cost[i][k] + dist[k][j])
42                {
43                    dist[i][j] = cost[i][k] + dist[k][j];
44                    next[i][j] = k;
45                }
46            }
47        }
48    }
49
50    printf("\nRouting Tables:\n");
51
52    for(i=0;i<n;i++)
53    {
54        printf("\nRouter %d\n",i);
55
56        printf("Destination\tCost\tNext Hop\n");
57
58        for(j=0;j<n;j++)
59        {
60            printf("%d\t%d\t%d\n",j,dist[i][j],next[i][j]);
61        }
62    }
63
64    return 0;
65 }
```

6. FTP using C

Filename: FTP/ftp_cli.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7 #include <fcntl.h>
8 #include <sys/stat.h>
9
10 #define PORT 9090
11 #define BUFFER 4096
12
13 ssize_t send_all(int fd, const void *buf, size_t len) {
14     size_t total = 0;
15     const char *p = buf;
16
17     while (total < len) {
18         ssize_t sent = send(fd, p + total, len - total, 0);
19         if (sent <= 0) return -1;
20         total += sent;
21     }
22     return total;
23 }
24
25 ssize_t recv_line(int fd, char *buffer, size_t max_len) {
26     size_t total = 0;
27     char c;
28
29     while (total < max_len - 1) {
30         ssize_t n = recv(fd, &c, 1, 0);
31         if (n <= 0) return -1;
32
33         buffer[total++] = c;
34
35         if (total >= 2 &&
36             buffer[total - 2] == '\r' &&
37             buffer[total - 1] == '\n')
38             break;
39     }
40
41     buffer[total] = '\0';
42     return total;
43 }
44
45 int main() {
46
47     int sock = socket(AF_INET, SOCK_STREAM, 0);
48
49     struct sockaddr_in addr = {0};
50     addr.sin_family = AF_INET;
51     addr.sin_port = htons(PORT);
52     inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
53
54     connect(sock, (struct sockaddr*)&addr, sizeof(addr));
55
56     char command[BUFFER];
57     char buffer[BUFFER];
58
59     while (1) {
60
61         printf("ftp> ");
62         fgets(command, BUFFER, stdin);
63
64         /* PUT */
65         if (strncmp(command, "put ", 4) == 0) {
66
67             char filename[256];
68             sscanf(command, "put %255s", filename);
```

```

69
70     struct stat st;
71     if (stat(filename, &st) < 0) {
72         printf("File not found\n");
73         continue;
74     }
75
76     sprintf(buffer, "PUT %s %ld\r\n", filename, st.st_size);
77     send_all(sock, buffer, strlen(buffer));
78
79     if (recv_line(sock, buffer, BUFFER) <= 0)
80         continue;
81
82     if (strncmp(buffer, "READY", 5) != 0)
83         continue;
84
85     int fd = open(filename, O_RDONLY);
86     ssize_t r;
87     while ((r = read(fd, buffer, BUFFER)) > 0)
88         send_all(sock, buffer, r);
89     close(fd);
90
91     recv_line(sock, buffer, BUFFER);
92     printf("%s", buffer);
93 }
94
95 /* GET */
96 else if (strncmp(command, "get ", 4) == 0) {
97
98     char filename[256];
99     sscanf(command, "get %255s", filename);
100
101     sprintf(buffer, "GET %s\r\n", filename);
102     send_all(sock, buffer, strlen(buffer));
103
104     if (recv_line(sock, buffer, BUFFER) <= 0)
105         continue;
106
107     long size;
108     if (sscanf(buffer, "SIZE %ld", &size) != 1) {
109         printf("Error retrieving file\n");
110         continue;
111     }
112
113     send_all(sock, "READY\r\n", 7);
114
115     int fd = open(filename,
116                   O_WRONLY | O_CREAT | O_TRUNC, 0644);
117
118     long remaining = size;
119
120     while (remaining > 0) {
121         ssize_t r = recv(sock, buffer,
122                         remaining > BUFFER ? BUFFER : remaining, 0);
123         write(fd, buffer, r);
124         remaining -= r;
125     }
126
127     close(fd);
128     printf("Download complete\n");
129 }
130
131 /* LIST */
132 else if (strncmp(command, "list", 4) == 0) {
133
134     send_all(sock, "LIST\r\n", 6);
135
136     if (recv_line(sock, buffer, BUFFER) <= 0)
137         continue;
138
139     long size;
140     sscanf(buffer, "LISTING %ld", &size);
141

```

```

142     long remaining = size;
143
144     while (remaining > 0) {
145         ssize_t r = recv(sock, buffer,
146                         remaining > BUFFER ? BUFFER : remaining, 0);
147         fwrite(buffer, 1, r, stdout);
148         remaining -= r;
149     }
150 }
151
152 /* QUIT */
153 else if (strncmp(command, "quit", 4) == 0) {
154     send_all(sock, "QUIT\r\n", 6);
155     break;
156 }
157 }
158
159 close(sock);
160 return 0;
161 }

```

6. FTP using C

Filename: FTP/ftp_serv.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <sys/socket.h>
7 #include <dirent.h>
8 #include <sys/stat.h>
9 #include <fcntl.h>
10 #include <libgen.h> // basename()
11
12 #define PORT 9090
13 #define BUFFER 4096
14 #define STORAGE_DIR "uploads"
15
16 /* ----- Reliable send ----- */
17 ssize_t send_all(int fd, const void *buf, size_t len) {
18     size_t total = 0;
19     const char *p = buf;
20
21     while (total < len) {
22         ssize_t sent = send(fd, p + total, len - total, 0);
23         if (sent <= 0) return -1;
24         total += sent;
25     }
26     return total;
27 }
28
29 /* ----- Read one CRLF terminated line ----- */
30 ssize_t recv_line(int fd, char *buffer, size_t max_len) {
31     size_t total = 0;
32     char c;
33
34     while (total < max_len - 1) {
35         ssize_t n = recv(fd, &c, 1, 0);
36         if (n <= 0) return -1;
37
38         buffer[total++] = c;
39
40         if (total >= 2 &&
41             buffer[total - 2] == '\r' &&
42             buffer[total - 1] == '\n')
43             break;
44     }
45
46     buffer[total] = '\0';
47     return total;
48 }
49
50 /* ----- Ensure uploads directory exists ----- */
51 void ensure_storage_dir() {
52     struct stat st = {0};
53     if (stat(STORAGE_DIR, &st) == -1) {
54         mkdir(STORAGE_DIR, 0755);
55     }
56 }
57
58 /* ----- Handle one client ----- */
59 void handle_client(int client_fd) {
60
61     char buffer[BUFFER];
62
63     while (1) {
64
65         if (recv_line(client_fd, buffer, BUFFER) <= 0)
66             break;
67
68         printf("Received: %s", buffer);
```

```

69
70  /* ----- PUT ----- */
71  if (strncmp(buffer, "PUT", 3) == 0) {
72
73      char filename[256];
74      size_t filesize;
75
76      sscanf(buffer, "PUT %255s %zu", filename, &filesize);
77
78      /* Strip path components */
79      char *base = basename(filename);
80
81      char safe_path[512];
82      snprintf(safe_path, sizeof(safe_path),
83              "%s/%s", STORAGE_DIR, base);
84
85      printf("Saving to: %s (%zu bytes)\n",
86             safe_path, filesize);
87
88      send_all(client_fd, "READY\r\n", 7);
89
90      int fd = open(safe_path,
91                  O_WRONLY | O_CREAT | O_TRUNC, 0644);
92      if (fd < 0) {
93          perror("open");
94          send_all(client_fd, "ERR\r\n", 5);
95          continue;
96      }
97
98      size_t remaining = filesize;
99
100     while (remaining > 0) {
101         ssize_t r = recv(client_fd, buffer,
102                          remaining > BUFFER ? BUFFER : remaining, 0);
103         if (r <= 0) break;
104
105         write(fd, buffer, r);
106         remaining -= r;
107     }
108
109     close(fd);
110
111     printf("Upload complete\n");
112     send_all(client_fd, "OK\r\n", 4);
113 }
114
115 /* ----- GET ----- */
116 else if (strncmp(buffer, "GET", 3) == 0) {
117
118     char filename[256];
119     sscanf(buffer, "GET %255s", filename);
120
121     char *base = basename(filename);
122
123     char safe_path[512];
124     snprintf(safe_path, sizeof(safe_path),
125             "%s/%s", STORAGE_DIR, base);
126
127     struct stat st;
128     if (stat(safe_path, &st) < 0) {
129         send_all(client_fd, "ERR\r\n", 5);
130         continue;
131     }
132
133     char header[64];
134     sprintf(header, "SIZE %ld\r\n", st.st_size);
135     send_all(client_fd, header, strlen(header));
136
137     if (recv_line(client_fd, buffer, BUFFER) <= 0)
138         continue;
139
140     int fd = open(safe_path, O_RDONLY);
141     if (fd < 0)

```



```

142         continue;
143
144         ssize_t r;
145         while ((r = read(fd, buffer, BUFFER)) > 0) {
146             send_all(client_fd, buffer, r);
147         }
148
149         close(fd);
150
151         printf("Download sent\n");
152     }
153
154     /* ----- LIST ----- */
155     else if (strncmp(buffer, "LIST", 4) == 0) {
156
157         DIR *dir = opendir(STORAGE_DIR);
158         if (!dir) continue;
159
160         char listing[8192] = {0};
161         size_t len = 0;
162
163         struct dirent *entry;
164         while ((entry = readdir(dir)) != NULL) {
165
166             if (strcmp(entry->d_name, ".") == 0 ||
167                 strcmp(entry->d_name, "..") == 0)
168                 continue;
169
170             size_t name_len = strlen(entry->d_name);
171
172             if (len + name_len + 2 >= sizeof(listing))
173                 break;
174
175             strcpy(listing + len, entry->d_name);
176             len += name_len;
177             listing[len++] = '\n';
178         }
179
180         closedir(dir);
181
182         char header[64];
183         sprintf(header, "LISTING %ld\r\n", len);
184
185         send_all(client_fd, header, strlen(header));
186         send_all(client_fd, listing, len);
187
188         printf("Listing sent\n");
189     }
190
191     /* ----- QUIT ----- */
192     else if (strncmp(buffer, "QUIT", 4) == 0) {
193         break;
194     }
195
196     else {
197         send_all(client_fd, "UNKNOWN\r\n", 9);
198     }
199 }
200
201 close(client_fd);
202 printf("Client disconnected\n");
203 }
204
205 /* ----- Main ----- */
206 int main() {
207
208     ensure_storage_dir();
209
210     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
211     if (server_fd < 0) {
212         perror("socket");
213         exit(1);
214     }

```

```

215
216     int opt = 1;
217     setsockopt(server_fd, SOL_SOCKET,
218                SO_REUSEADDR, &opt, sizeof(opt));
219
220     struct sockaddr_in addr = {0};
221     addr.sin_family = AF_INET;
222     addr.sin_addr.s_addr = INADDR_ANY;
223     addr.sin_port = htons(PORT);
224
225     if (bind(server_fd,
226              (struct sockaddr*)&addr,
227              sizeof(addr)) < 0) {
228         perror("bind");
229         exit(1);
230     }
231
232     if (listen(server_fd, 5) < 0) {
233         perror("listen");
234         exit(1);
235     }
236
237     printf("FTP Server running on port %d\n", PORT);
238
239     while (1) {
240
241         int client_fd = accept(server_fd, NULL, NULL);
242         if (client_fd < 0) {
243             perror("accept");
244             continue;
245         }
246
247         printf("Client connected\n");
248         handle_client(client_fd);
249     }
250
251     close(server_fd);
252     return 0;
253 }

```

7. Leaky bucket congestion control

Filename: leaky-bucket/leakybucket.c

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main()
5 {
6     int bucket_size, output_rate, packets, stored = 0;
7     int incoming;
8
9     printf("Enter bucket capacity: ");
10    scanf("%d",&bucket_size);
11
12    printf("Enter output rate: ");
13    scanf("%d",&output_rate);
14
15    printf("Enter number of time steps: ");
16    scanf("%d",&packets);
17
18    for(int i = 1; i <= packets; i++)
19    {
20        printf("\nTime %d\n",i);
21
22        printf("Enter incoming packets: ");
23        scanf("%d",&incoming);
24
25        if(incoming + stored <= bucket_size)
26        {
27            stored += incoming;
28        }
29        else
30        {
31            int dropped = (incoming + stored) - bucket_size;
32            stored = bucket_size;
33
34            printf("Bucket overflow! Dropped packets = %d\n",dropped);
35        }
36
37        printf("Packets in bucket: %d\n",stored);
38
39        int sent;
40
41        if(stored < output_rate)
42            sent = stored;
43        else
44            sent = output_rate;
45
46        stored -= sent;
47
48        printf("Packets sent: %d\n",sent);
49        printf("Packets remaining in bucket: %d\n",stored);
50
51        sleep(1);
52    }
53
54    return 0;
55 }
```